

100 C# Practice Problems

1 Beginner Level (1–30)

Basic Input / Output

1. Print **Hello World**.
2. Print your **name and age**.
3. Take user input and print it.
4. Add two numbers.
5. Swap two numbers.

Conditions

6. Check if a number is **even or odd**.
7. Find **largest of 3 numbers**.
8. Check if a number is **positive or negative**.
9. Check **voting eligibility (age ≥ 18)**.
10. Check if a number is **divisible by 5 and 11**.

Loops

11. Print numbers **1–100**.
12. Print **even numbers from 1–50**.
13. Print multiplication table of a number.
14. Find **sum of numbers 1–N**.
15. Count digits in a number.

Basic Math

16. Find factorial of a number.
17. Check **prime number**.
18. Reverse a number.
19. Check **palindrome number**.
20. Find sum of digits.

Patterns

- 21. Print star triangle.
- 22. Print reverse star triangle.
- 23. Print number pyramid.
- 24. Print square star pattern.
- 25. Print diamond pattern.

Simple Programs

- 26. Convert **Celsius** → **Fahrenheit**.
 - 27. Calculate **simple interest**.
 - 28. Find **area of circle**.
 - 29. Find **area of rectangle**.
 - 30. Generate **random number guessing game**.
-

2 Basic Logic (31–60)

Arrays

- 31. Print elements of array.
- 32. Find **largest element** in array.
- 33. Find **smallest element**.
- 34. Calculate **sum of array**.
- 35. Calculate **average of array**.

Array Operations

- 36. Reverse an array.
- 37. Sort array ascending.
- 38. Sort array descending.
- 39. Find **second largest element**.
- 40. Remove duplicates.

Searching

- 41. Linear search.
- 42. Binary search.

Strings

- 43. Reverse a string.
- 44. Count vowels in string.
- 45. Check palindrome string.
- 46. Convert string to uppercase.
- 47. Count words in sentence.

String Logic

- 48. Remove spaces from string.
- 49. Find longest word.
- 50. Check if two strings are anagrams.

Lists

- 51. Store numbers in List<int>.
- 52. Find maximum in list.
- 53. Remove item from list.
- 54. Sort list.
- 55. Find duplicate numbers.

Practice Programs

- 56. Simple calculator.
- 57. Password strength checker.
- 58. Temperature converter.
- 59. Number guessing game.
- 60. Student grade calculator.

3 Intermediate Level (61–85)

Methods

- 61. Method to calculate factorial.
- 62. Method to check prime number.
- 63. Method to reverse string.
- 64. Method to find maximum number.
- 65. Method returning average.

Classes / Objects

- 66. Create **Car class**.
- 67. Create **Student class**.
- 68. Create **BankAccount class**.
- 69. Create **Rectangle class** with area method.
- 70. Create **Employee class**.

OOP Concepts

- 71. Demonstrate **encapsulation**.
- 72. Demonstrate **inheritance**.
- 73. Demonstrate **method overloading**.
- 74. Demonstrate **method overriding**.
- 75. Create **Animal → Dog/Cat hierarchy**.

Collections

- 76. Use **Dictionary** to store contacts.
- 77. Count frequency of numbers in array.
- 78. Count frequency of characters in string.
- 79. Find intersection of two arrays.
- 80. Merge two arrays.

File Handling

- 81. Write text to file.
 - 82. Read text from file.
 - 83. Count lines in file.
 - 84. Append data to file.
 - 85. Build simple **notes file system**.
-

Advanced Level (86–100)

LINQ

- 86. Filter even numbers using LINQ.
- 87. Find max value using LINQ.
- 88. Sort list using LINQ.
- 89. Group words by length.
- 90. Remove duplicates with LINQ.

Algorithms

- 91. Bubble sort.
- 92. Selection sort.
- 93. Fibonacci sequence.
- 94. Find missing number in array.
- 95. Detect duplicate numbers.

Real Projects

- 96. **Console To-Do List**
- 97. **Expense Tracker**
- 98. **Student Management System**
- 99. **Library Management System**
- 100. **Contact Book**

Best Sites to Practice

You can solve similar problems on:

- [HackerRank](#)
- [LeetCode](#)
- [Codewars](#)

Pro Tip for Faster Learning

Follow this rule daily:

- Solve **3 problems**
- Read **1 concept**
- Improve **1 old program**

C# Problems 1–10 (Basics)

1. Hello World

Definition:

The first program used to understand how a program runs.

Code

```
using System;
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        Console.WriteLine("Hello World");
```

```
    }
```

```
}
```

Interview Tip

- Main() is the entry point of every C# program.

2. Print Name and Age

Definition:

Using Console.WriteLine() to display text.

Code

```
using System;
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

C# Code Practice BASIC to ADVANCE

```
Console.WriteLine("Name: John");  
Console.WriteLine("Age: 25");  
}  
}
```

Interview Tip

- Console.WriteLine() prints output and moves to next line.
-

3. Take User Input

Definition:

Using Console.ReadLine() to get input from user.

Code

```
using System;  
  
class Program  
{  
    static void Main()  
    {  
        Console.Write("Enter your name: ");  
        string name = Console.ReadLine();  
  
        Console.WriteLine("Hello " + name);  
    }  
}
```

Interview Tip

- Console.ReadLine() returns **string**.
-

4. Add Two Numbers

Definition:

Basic arithmetic operations.

Code

```
using System;
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        int a = 5;
```

```
        int b = 10;
```

```
        int sum = a + b;
```

```
        Console.WriteLine("Sum = " + sum);
```

```
    }
```

```
}
```

Interview Tip

- Operators: + - * / %.

5. Swap Two Numbers

Definition:

Exchange values of variables.

Code

```
using System;
```

C# Code Practice BASIC to ADVANCE

```
class Program
{
    static void Main()
    {
        int a = 5;
        int b = 10;

        int temp = a;
        a = b;
        b = temp;

        Console.WriteLine("A = " + a);
        Console.WriteLine("B = " + b);
    }
}
```

Interview Tip

- Often asked to swap **without temporary variable**.

6. Check Even or Odd

Definition:

Use modulus operator %.

Code

```
using System;
```

```
class Program
```

```
{
```

```
static void Main()
{
    int num = 10;

    if (num % 2 == 0)
        Console.WriteLine("Even");
    else
        Console.WriteLine("Odd");
}
}
```

Interview Tip

- % returns remainder.
-

7. Largest of Three Numbers

Definition:

Comparison using if.

Code

```
using System;
```

```
class Program
```

```
{
    static void Main()
    {
        int a = 10, b = 20, c = 15;

        if (a > b && a > c)
```

```
        Console.WriteLine("A is largest");
    else if (b > c)
        Console.WriteLine("B is largest");
    else
        Console.WriteLine("C is largest");
    }
}
```

Interview Tip

- Logical operators: && and ||.

8. Positive or Negative Number

Code

```
using System;

class Program
{
    static void Main()
    {
        int num = -5;

        if (num > 0)
            Console.WriteLine("Positive");
        else if (num < 0)
            Console.WriteLine("Negative");
        else
            Console.WriteLine("Zero");
    }
}
```

```
}  
}
```

9. Voting Eligibility

Definition:

Conditional logic.

Code

```
using System;
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        int age = 20;
```

```
        if (age >= 18)
```

```
            Console.WriteLine("Eligible to vote");
```

```
        else
```

```
            Console.WriteLine("Not eligible");
```

```
    }
```

```
}
```

10. Divisible by 5 and 11

Code

```
using System;
```

```
class Program
{
    static void Main()
    {
        int num = 55;

        if (num % 5 == 0 && num % 11 == 0)
            Console.WriteLine("Divisible by 5 and 11");
        else
            Console.WriteLine("Not divisible");
    }
}
```

C# Problems 11–15 (Loops)

11. Print Numbers 1–100

```
for(int i = 1; i <= 100; i++)
{
    Console.WriteLine(i);
}
```

Interview Tip

- for(initialization; condition; increment).
-

12. Even Numbers 1–50

```
for(int i = 2; i <= 50; i += 2)
{
    Console.WriteLine(i);
}
```

```
}
```

13. Multiplication Table

```
int num = 5;

for(int i = 1; i <= 10; i++)
{
    Console.WriteLine(num + " x " + i + " = " + num * i);
}
```

14. Sum 1–N

```
int n = 10;
int sum = 0;

for(int i = 1; i <= n; i++)
{
    sum += i;
}

Console.WriteLine(sum);
```

15. Count Digits

```
int num = 12345;
int count = 0;

while(num != 0)
```

```
{  
    num /= 10;  
    count++;  
}
```

```
Console.WriteLine(count);
```

C# Problems 16–20 (Math Logic)

16. Factorial

```
int num = 5;  
int fact = 1;  
  
for(int i = 1; i <= num; i++)  
{  
    fact *= i;  
}
```

```
Console.WriteLine(fact);
```

17. Prime Number

```
int num = 7;  
bool isPrime = true;  
  
for(int i = 2; i <= num/2; i++)  
{  
    if(num % i == 0)
```



```
{  
    isPrime = false;  
    break;  
}  
}
```

```
Console.WriteLine(isPrime ? "Prime" : "Not Prime");
```

18. Reverse Number

```
int num = 1234;  
int reverse = 0;  
  
while(num != 0)  
{  
    int digit = num % 10;  
    reverse = reverse * 10 + digit;  
    num /= 10;  
}
```

```
Console.WriteLine(reverse);
```

19. Palindrome Number

```
int num = 121;  
int original = num;  
int reverse = 0;
```

```
while(num != 0)
{
    reverse = reverse * 10 + num % 10;
    num /= 10;
}

if(original == reverse)
    Console.WriteLine("Palindrome");
else
    Console.WriteLine("Not Palindrome");
```

20. Sum of Digits

```
int num = 1234;
int sum = 0;

while(num != 0)
{
    sum += num % 10;
    num /= 10;
}
```

```
Console.WriteLine(sum);
```

C# Problems 21–25 (Patterns)

21. Star Triangle

```
for(int i = 1; i <= 5; i++)
```

```
{  
    for(int j = 1; j <= i; j++)  
    {  
        Console.Write("*");  
    }  
    Console.WriteLine();  
}
```

22. Reverse Triangle

```
for(int i = 5; i >= 1; i--)  
{  
    for(int j = 1; j <= i; j++)  
    {  
        Console.Write("*");  
    }  
    Console.WriteLine();  
}
```

23. Number Pyramid

```
for(int i = 1; i <= 5; i++)  
{  
    for(int j = 1; j <= i; j++)  
    {  
        Console.Write(i + " ");  
    }  
    Console.WriteLine();  
}
```

```
}
```

24. Square Pattern

```
for(int i = 1; i <= 5; i++)  
{  
    for(int j = 1; j <= 5; j++)  
    {  
        Console.Write("* ");  
    }  
    Console.WriteLine();  
}
```

25. Diamond Pattern

```
int n = 5;  
  
for(int i = 1; i <= n; i++)  
{  
    Console.Write(new string(' ', n-i));  
    Console.WriteLine(new string('*', 2*i-1));  
}  
  
for(int i = n-1; i >= 1; i--)  
{  
    Console.Write(new string(' ', n-i));  
    Console.WriteLine(new string('*', 2*i-1));  
}
```

C# Problems 26–30 (Small Programs)

26. Celsius → Fahrenheit

```
double c = 30;  
double f = (c * 9 / 5) + 32;  
Console.WriteLine(f);
```

27. Simple Interest

```
double p = 1000;  
double r = 5;  
double t = 2;  
  
double si = (p * r * t) / 100;  
  
Console.WriteLine(si);
```

28. Area of Circle

```
double r = 5;  
double area = Math.PI * r * r;  
  
Console.WriteLine(area);
```

29. Area of Rectangle

```
int length = 10;  
int width = 5;  
  
int area = length * width;
```

```
Console.WriteLine(area);
```

30. Random Number Guessing Game

```
Random rand = new Random();
```

```
int number = rand.Next(1, 10);
```

```
Console.Write("Guess number: ");
```

```
int guess = int.Parse(Console.ReadLine());
```

```
if(guess == number)
```

```
    Console.WriteLine("Correct!");
```

```
else
```

```
    Console.WriteLine("Wrong! Number was " + number);
```

Interview Preparation Questions (Important)

You should know answers for:

- 1 What is **CLR**?
- 2 What is **.NET Framework**?
- 3 Difference between **int** and **double**
- 4 What is **Console.ReadLine()**?
- 5 Difference between **for loop** and **while loop**
- 6 What is **modulus operator**?
- 7 What is **array**?
- 8 What is **class** and **object**?
- 9 What is **method**?
- 10 What is **exception handling**?

31. Print Elements of an Array

Explanation:

An **array** stores multiple values of the same type in one variable.

Code

```
using System;
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        int[] numbers = {1,2,3,4,5};
```

```
        foreach(int num in numbers)
```

```
        {
```

```
            Console.WriteLine(num);
```

```
        }
```

```
    }
```

```
}
```

Interview Tip

- Arrays have **fixed size** in C#.

32. Find Largest Element in Array

Code

```
using System;
```

```
class Program
```

```
{
    static void Main()
    {
        int[] arr = {10, 5, 20, 8};

        int max = arr[0];

        foreach(int num in arr)
        {
            if(num > max)
                max = num;
        }

        Console.WriteLine("Largest: " + max);
    }
}
```

33. Find Smallest Element

Code

```
using System;

class Program
{
    static void Main()
    {
        int[] arr = {10, 5, 20, 8};
```



```
int min = arr[0];

foreach(int num in arr)
{
    if(num < min)
        min = num;
}

Console.WriteLine("Smallest: " + min);
}
}
```

34. Sum of Array

Explanation:

Adding all elements stored in an array.

Code

```
using System;

class Program
{
    static void Main()
    {
        int[] arr = {1,2,3,4,5};
        int sum = 0;
```

```
foreach(int num in arr)
{
    sum += num;
}

Console.WriteLine(sum);
}
```

35. Average of Array

Code

```
using System;

class Program
{
    static void Main()
    {
        int[] arr = {2,4,6,8};
        int sum = 0;

        foreach(int num in arr)
        {
            sum += num;
        }

        double avg = (double)sum / arr.Length;
```

```
    Console.WriteLine(avg);  
}  
}
```

C# Problems 36–40 (Array Operations)

36. Reverse an Array

Explanation:

Reverse means last element becomes first.

Code

```
using System;  
  
class Program  
{  
    static void Main()  
    {  
        int[] arr = {1,2,3,4};  
  
        Array.Reverse(arr);  
  
        foreach(int num in arr)  
            Console.WriteLine(num);  
    }  
}
```

Interview Tip

- `Array.Reverse()` is built-in.

37. Sort Array Ascending

using System;

class Program

{

static void Main()

{

int[] arr = {5,1,4,2};

Array.Sort(arr);

foreach(int num in arr)

Console.WriteLine(num);

}

}

38. Sort Array Descending

using System;

class Program

{

static void Main()

{

int[] arr = {5,1,4,2};

```
Array.Sort(arr);  
Array.Reverse(arr);  
  
foreach(int num in arr)  
    Console.WriteLine(num);  
}  
}
```

39. Second Largest Number

```
using System;  
  
class Program  
{  
    static void Main()  
    {  
        int[] arr = {10,5,20,8};  
  
        Array.Sort(arr);  
  
        Console.WriteLine(arr[arr.Length-2]);  
    }  
}
```

40. Remove Duplicates

```
using System;  
using System.Linq;
```

```
class Program
{
    static void Main()
    {
        int[] arr = {1,2,2,3,3,4};

        var result = arr.Distinct();

        foreach(var num in result)
            Console.WriteLine(num);
    }
}
```

C# Problems 41–42 (Searching)

41. Linear Search

Explanation:

Check each element until found.

using System;

```
class Program
{
    static void Main()
    {
        int[] arr = {10,20,30};
        int target = 20;
```

```
foreach(int num in arr)
{
    if(num == target)
    {
        Console.WriteLine("Found");
        return;
    }
}

Console.WriteLine("Not Found");
}
```

42. Binary Search

Explanation:

Works only on **sorted arrays**.

using System;

```
class Program
{
    static void Main()
    {
        int[] arr = {1,3,5,7};

        int index = Array.BinarySearch(arr,5);
```

```
        Console.WriteLine(index);  
    }  
}
```

C# Problems 43–47 (String Operations)

43. Reverse String

```
using System;
```

```
class Program  
{  
    static void Main()  
    {  
        string str = "hello";  
  
        char[] arr = str.ToCharArray();  
        Array.Reverse(arr);  
  
        Console.WriteLine(new string(arr));  
    }  
}
```

44. Count Vowels

```
using System;
```

```
class Program
```

```
{  
    static void Main()  
    {  
        string str = "education";  
        int count = 0;  
  
        foreach(char c in str.ToLower())  
        {  
            if("aeiou".Contains(c))  
                count++;  
        }  
  
        Console.WriteLine(count);  
    }  
}
```

45. Palindrome String

```
using System;  
  
class Program  
{  
    static void Main()  
    {  
        string str = "level";  
  
        char[] arr = str.ToCharArray();
```

```
Array.Reverse(arr);

string rev = new string(arr);

Console.WriteLine(str == rev ? "Palindrome" : "Not");
}
}
```

46. Convert String to Uppercase

```
string text = "hello world";

Console.WriteLine(text.ToUpper());
```

47. Count Words in Sentence

```
using System;

class Program
{
    static void Main()
    {
        string sentence = "C sharp programming language";

        int words = sentence.Split(' ').Length;

        Console.WriteLine(words);
    }
}
```

```
}
```

C# Problems 48–50 (String Logic)

48. Remove Spaces

```
string text = "hello world";
```

```
string result = text.Replace(" ","");
```

```
Console.WriteLine(result);
```

49. Longest Word

```
using System;
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        string sentence = "C sharp programming language";
```

```
        string[] words = sentence.Split(' ');
```

```
        string longest = "";
```

```
        foreach(string w in words)
```

```
        {
```

```
            if(w.Length > longest.Length)
```

```
        longest = w;
    }

    Console.WriteLine(longest);
}
}
```

50. Anagram Check

```
using System;

class Program
{
    static void Main()
    {
        string a = "listen";
        string b = "silent";

        char[] arr1 = a.ToCharArray();
        char[] arr2 = b.ToCharArray();

        Array.Sort(arr1);
        Array.Sort(arr2);

        Console.WriteLine(new string(arr1) == new string(arr2));
    }
}
```

C# Problems 51–55 (Lists)

51. List Example

```
using System.Collections.Generic;
```

```
List<int> numbers = new List<int>{1,2,3};
```

```
foreach(int n in numbers)
```

```
    Console.WriteLine(n);
```

52. Max in List

```
using System.Linq;
```

```
List<int> nums = new List<int>{1,5,3};
```

```
Console.WriteLine(nums.Max());
```

53. Remove Item

```
List<int> nums = new List<int>{1,2,3};
```

```
nums.Remove(2);
```

54. Sort List

```
List<int> nums = new List<int>{5,1,3};
```

```
nums.Sort();
```

55. Find Duplicate Numbers

```
using System.Linq;

List<int> nums = new List<int>{1,2,2,3,3};

var duplicates = nums.GroupBy(x => x)
    .Where(g => g.Count() > 1)
    .Select(g => g.Key);

foreach(var d in duplicates)
    Console.WriteLine(d);
```

C# Problems 56–60 (Mini Programs)

56. Simple Calculator

```
int a = 10;
int b = 5;

Console.WriteLine("Add: " + (a+b));
Console.WriteLine("Sub: " + (a-b));
Console.WriteLine("Mul: " + (a*b));
Console.WriteLine("Div: " + (a/b));
```

57. Password Strength Checker

```
string password = "Test123";

if(password.Length >= 8)
```

```
    Console.WriteLine("Strong");  
else  
    Console.WriteLine("Weak");
```

58. Temperature Converter

```
double f = 98;  
  
double c = (f - 32) * 5/9;  
  
Console.WriteLine(c);
```

59. Number Guessing Game

```
Random rand = new Random();  
  
int number = rand.Next(1,10);  
  
Console.Write("Guess: ");  
int guess = int.Parse(Console.ReadLine());  
  
if(guess == number)  
    Console.WriteLine("Correct");  
else  
    Console.WriteLine("Wrong");
```

60. Student Grade Calculator

```
int marks = 75;
```

C# Code Practice BASIC to ADVANCE

```
if(marks >= 90)
    Console.WriteLine("A");
else if(marks >= 75)
    Console.WriteLine("B");
else if(marks >= 60)
    Console.WriteLine("C");
else
    Console.WriteLine("Fail");
```


61. Method to Calculate Factorial

Explanation

A **method** is a reusable block of code.

using System;

class Program

{

static int Factorial(int n)

{

int result = 1;

for(int i = 1; i <= n; i++)

result *= i;

return result;

}

static void Main()

{

Console.WriteLine(Factorial(5));

}

}

 Interview Tip

Know **difference between void method and return method**.

62. Method to Check Prime Number

static bool IsPrime(int n)

{

```
if(n <= 1) return false;

for(int i=2;i<=n/2;i++)
    if(n % i == 0)
        return false;

return true;
}
```

63. Method to Reverse String

```
static string Reverse(string text)
{
    char[] arr = text.ToCharArray();
    Array.Reverse(arr);
    return new string(arr);
}
```

64. Method to Find Maximum

```
static int Max(int a,int b)
{
    return a > b ? a : b;
}
```

65. Method Returning Average

```
static double Average(int[] numbers)
{

```

```
int sum = 0;

foreach(int n in numbers)

    sum += n;

return (double)sum / numbers.Length;
}
```

66–70 Classes and Objects

66. Create Car Class

Explanation

A **class** is a blueprint for objects.

```
class Car
{
    public string Name;
    public int Speed;
}
```

Usage:

```
Car car = new Car();
car.Name = "BMW";
car.Speed = 200;
```

67. Student Class

```
class Student
{
    public string Name;
```

```
public int Age;

public void Display()
{
    Console.WriteLine(Name + " " + Age);
}
}
```

68. BankAccount Class

```
class BankAccount
{
    public double Balance;

    public void Deposit(double amount)
    {
        Balance += amount;
    }

    public void Withdraw(double amount)
    {
        Balance -= amount;
    }
}
```



Interview Tip

Understand **encapsulation in banking systems**.

69. Rectangle Class

```
class Rectangle
{
    public int Length;
    public int Width;

    public int Area()
    {
        return Length * Width;
    }
}
```

70. Employee Class

```
class Employee
{
    public string Name;
    public double Salary;

    public void Display()
    {
        Console.WriteLine(Name + " earns " + Salary);
    }
}
```

71-75 OOP Concepts

71. Encapsulation

```
class Person
{
    private int age;

    public int Age
    {
        get { return age; }
        set { age = value; }
    }
}
```

Explanation

Encapsulation hides internal data.

72. Inheritance

```
class Animal
{
    public void Eat()
    {
        Console.WriteLine("Eating...");
    }
}

class Dog : Animal
{

```

```
public void Bark()
{
    Console.WriteLine("Barking...");
}
}
```

73. Method Overloading

```
class MathUtil
{
    public int Add(int a,int b)
    {
        return a + b;
    }

    public double Add(double a,double b)
    {
        return a + b;
    }
}
```

74. Method Overriding

```
class Animal
{
    public virtual void Speak()
    {
        Console.WriteLine("Animal sound");
    }
}
```

```
    }  
}  
  
class Dog : Animal  
{  
    public override void Speak()  
    {  
        Console.WriteLine("Bark");  
    }  
}
```

75. Animal Hierarchy

```
class Animal  
{  
    public void Move()  
    {  
        Console.WriteLine("Moving");  
    }  
}
```

```
class Cat : Animal  
{  
    public void Meow()  
    {  
        Console.WriteLine("Meow");  
    }  
}
```



```
}
```

76–80 Collections & Dictionaries

76. Dictionary Contact List

```
Dictionary<string,string> contacts = new Dictionary<string,string>();
```

```
contacts["John"] = "12345";
```

```
contacts["Alice"] = "67890";
```

```
Console.WriteLine(contacts["John"]);
```

77. Count Frequency in Array

```
using System.Linq;
```

```
int[] arr = {1,2,2,3,3,3};
```

```
var freq = arr.GroupBy(x => x)
```

```
    .Select(g => new {Number = g.Key, Count = g.Count()});
```

```
foreach(var item in freq)
```

```
    Console.WriteLine(item.Number + " : " + item.Count);
```

78. Character Frequency

```
string text = "hello";
```

```
var result = text.GroupBy(c => c);
```

```
foreach(var g in result)
    Console.WriteLine(g.Key + " : " + g.Count());
```

79. Intersection of Two Arrays

```
int[] a = {1,2,3};
```

```
int[] b = {2,3,4};
```

```
var result = a.Intersect(b);
```

```
foreach(int n in result)
```

```
    Console.WriteLine(n);
```

80. Merge Two Arrays

```
int[] a = {1,2};
```

```
int[] b = {3,4};
```

```
var result = a.Concat(b);
```

```
foreach(var n in result)
```

```
    Console.WriteLine(n);
```

81–85 File Handling

81. Write to File

```
using System.IO;
```

```
File.WriteAllText("test.txt","Hello File");
```

82. Read File

```
string text = File.ReadAllText("test.txt");
```

```
Console.WriteLine(text);
```

83. Count Lines in File

```
int lines = File.ReadAllLines("test.txt").Length;
```

```
Console.WriteLine(lines);
```

84. Append Data to File

```
File.AppendAllText("test.txt","\nNew Line");
```

85. Simple Notes System

```
Console.Write("Write note: ");
```

```
string note = Console.ReadLine();
```

```
File.AppendAllText("notes.txt", note + "\n");
```

86–90 LINQ (Important for Interviews)

86. Filter Even Numbers

```
using System.Linq;
```

```
int[] nums = {1,2,3,4,5};
```

```
var evens = nums.Where(n => n % 2 == 0);
```

```
foreach(var n in evens)
    Console.WriteLine(n);
```

87. Find Maximum

```
int max = nums.Max();
```

88. Sort Using LINQ

```
var sorted = nums.OrderBy(n => n);
```

89. Group Words by Length

```
string[] words = {"cat","dog","tiger"};
```

```
var groups = words.GroupBy(w => w.Length);
```

90. Remove Duplicates

```
var unique = nums.Distinct();
```

91–95 Algorithms

91. Bubble Sort

```
for(int i=0;i<arr.Length;i++)
{
    for(int j=0;j<arr.Length-i-1;j++)
    {
```

```
    if(arr[j] > arr[j+1])
    {
        int temp = arr[j];
        arr[j] = arr[j+1];
        arr[j+1] = temp;
    }
}
}
```

92. Selection Sort

```
for(int i=0;i<arr.Length;i++)
{
    int min = i;

    for(int j=i+1;j<arr.Length;j++)
        if(arr[j] < arr[min])
            min = j;

    int temp = arr[min];
    arr[min] = arr[i];
    arr[i] = temp;
}
```

93. Fibonacci Sequence

```
int a=0,b=1;
```

```
for(int i=0;i<10;i++)  
{  
    Console.Write(a);  
    int temp = a+b;  
    a=b;  
    b=temp;  
}
```

94. Find Missing Number

```
int[] arr = {1,2,4,5};  
  
int sum = arr.Sum();  
int n = 5;  
  
int expected = n*(n+1)/2;  
  
Console.WriteLine(expected - sum);
```

95. Detect Duplicate Numbers

```
var dup = arr.GroupBy(x => x)  
    .Where(g => g.Count() > 1)  
    .Select(g => g.Key);
```

96–100 Real-World Mini Projects

96. Console To-Do List

Features:

- Add task
- Remove task
- Show tasks

Use:

List<string>

97. Expense Tracker

Features:

- Add expense
 - View expenses
 - Calculate total
-

98. Student Management System

Features:

- Add student
- Delete student
- Display students

Use:

List<Student>

99. Library Management System

Features:

- Add book
 - Borrow book
 - Return book
-

100. Contact Book

Features:

- Add contact
- Search contact
- Delete contact

Use:

Dictionary<string,string>

Final Interview Preparation Topics

You should understand:

1. **OOP Principles**
2. **LINQ queries**
3. **Collections vs Arrays**
4. **File handling**
5. **Exception handling**
6. **Delegates**
7. **Async/Await**
8. **Dependency Injection**
9. **.NET runtime**